Design and Implementation of a Flight Booking System Using Java Spring Boot and MySQL

University of Embu

C.I.T department:

Year 2026:

Developers:

- Lyean Njogu -B135/28569/2024
- Job Njagi -B135/31355/2024
- Brandon Ndeti -B135/25576/2024
- Beavon Ouko -B135/28335/2024

Table of content

Chapter One: Introduction

1.1 Background

1.2 Statement of the Problem

1.3 Justification

1.4 Objectives

1.5 Anticipated Output

Chapter Two: Literature Review

2.1 Overview of Existing Flight Booking Systems

2.2 Strengths and Weaknesses of Commercial Solutions

2.3 Review of Java Spring-Based Applications in Transport Management

2.4 Identification of Gaps in Local Adoption

2.5 Synthesis and Research Implications

Chapter Three: Materials and Methods

3.1 Requirement Analysis

3.2 System Design

3.3 Database Design (ER Diagrams, Schema)

3.4 Forms and Interfaces

3.5 Data Collection and Testing

Chapter Four: System Implementation

4.1 Development Environment

4.2 Implementation of Flights Module

4.3 Implementation of Bookings Module

4.4 Ticketing and Payment Integration

4.5 Admin Dashboard Customization

Chapter Five: Results and Discussion

5.1 System Outputs

5.2 Performance Evaluation

5.3 User Experience and Feedback

5.4 Comparison with Existing Solutions

Chapter Six: Conclusion and Recommendation

6.1 Summary of Findings

6.2 Conclusion

6.3 Recommendations for Future Work

Appendices

i. UML Diagrams (Client–Server Architecture, Sequence Diagrams)

ii. ER Diagrams (Database Design)

iii. Screenshots of Interfaces

iv. Test Cases and Results

LIST OF TABLES AND FIGURES

- Table 3.1: Functional Requirements of the Flyte System
- Table 3.2: Non-Functional Requirements
- Table 4.1: Database Schema for Flights Table
- Table 4.2: Database Schema for Bookings Tables
- Table 5.1: Test Cases and Expected Results

- Table 5.2: Comparison of Flyte vs Existing Flight Booking Systems
- Figure 2.1: Use Case Diagram for Flyte Booking System
- Figure 2.2: Class Diagram of System Architecture
- Figure 3.1: Entity Relationship Diagram (ERD) of Database Design
- Figure 3.2: Sequence Diagram for Booking Workflow
- Figure 4.1: Client–Server Architecture of Flyte System
- Figure 4.2: Admin Dashboard Interface Screenshot
- Figure 5.1: Sample Flight Search Results Page

Abstract

This project proposes the design and implementation of a flight booking system named **Flyte**, developed using **Java Spring Framework** in a **client–server architecture**. The system integrates two core modules: **Flights** and **Bookings**, with a customized admin interface for management. The aim is to provide a scalable, secure, and efficient solution for booking

and managing flights. The methodology involves requirement analysis, system design, coding, and testing. Anticipated outputs include a functional booking platform, improved efficiency in flight management, and a foundation for future enhancements such as payment integration and analytics.

Flyte Booking system

INTRODUCTION

1.1 Background

Flight booking systems are essential in modern transport management, enabling efficient scheduling, reservations, and customer service. Traditional manual booking methods are prone to inefficiencies and errors. With the rise of distributed systems, **Java Spring Framework** provides a robust environment for building scalable, secure, and modular applications. The **Flyte system** leverages Spring Boot for rapid development, Spring Data JPA for database management, and RESTful APIs for client–server communication. This project addresses gaps in local implementations by providing a tailored solution for small to medium-sized operators in Kenya and ensuring both stakeholder and customer requirements are met.

1.2 Statement of the problem

Air travelling in kenya is expanding rapidly with JKIA and WILSON serving as key hubs. Yet booking system and related digital platform often face userbility , reliability and transparency issues. Thus bringing hick ups to the country's greatest source of foreign exchange (tourism sector).

Existing booking systems are either too costly or lack customization for small operators. Few systems adopt lightweight, open-source frameworks that balance functionality with affordability. This project

seeks to bridge that gap by developing a **Java Spring-based client–server system** that is simple, efficient, and adaptable.

1.3 Justification

- Personal development: upskill ones knowledge on web development and related fields
- **Community Impact:** Provides affordable booking solutions for local operators.
- **Innovation:** Integrates modular design with RESTful APIs.
- **Contribution to Knowledge:** Demonstrates practical application of Java Spring in transport systems.
- **Economic Value:** Reduces operational costs and improves customer satisfaction.

1.4 Objective:

- **General Objective:** To design and implement a functional flight booking system using Java Spring Framework in a client–server architecture.

- **Specific Objectives:**

- To analyze requirements for a flight booking system.
- To design a modular system with Flights and Bookings services.

- To implement and test the system using Java Spring Boot and MySQL/SQLite.
- To develop RESTful APIs for client–server communication.
- To customize the admin interface for efficient management
- Implement M-pesa payment
- AI technical support(customer chatbot)

1.5 Anticipated output:

- A functional Flyte booking system.
- RESTful APIs for client–server communication.
- Admin dashboard with branding and usability enhancements.
- Documentation of system design and implementation.

LITERATURE REVIEW

Pros

- Highly scalable
- Strong integration
- User-friendly UI

Cons

- High cost
- Complexity
- Limited customization

ii. Local/Regional Solutions (Kenya Airways, Jambojet)

In contrast, regional airlines such as Kenya Airways and Jambojet provide web-based booking platforms tailored to local travelers. These systems allow customers to search flights, book tickets, check in online, and make payments using locally preferred methods such as M-Pesa. Jambojet, for instance, operates over 400 weekly flights, offering affordable and accessible services within East Africa.

Prons

- Payment integration
- Reliability

Cons

- Limited scalabiliy
- Customization gaps
- Dependance on internet connectivity

4. Review of Java Spring-Based Applications in Transport Management

4.1 Framework Advantages

Spring Boot supports microservices, enabling decomposition of complex booking systems. Spring Cloud Data Flow and Spring Cloud Stream facilitate real-time inventory updates, pricing optimisation, and event-driven architectures.

4.2 Documented Implementations

Case studies highlight Spring Boot's use in traffic management systems integrating GPS, maps, and AI for real-time monitoring. These architectural patterns translate directly to flight booking contexts.

4.3 Integration Capabilities

Spring's ecosystem supports heterogeneous connectivity with GDS APIs, NDC interfaces, and payment gateways. Containerisation via Docker and orchestration with Kubernetes enable scalable deployments, while Spring Security ensures compliance with PCI standards.

5. Identification of Gaps in Local systems

5.1 Content Gaps

Global platforms often fail to support local requirements, such as central billing for low-cost carriers.

5.2 Linguistic and Cultural Localisation

Language and cultural adaptation are essential for trust and conversion. Platforms underperform when localisation is treated as an afterthought.

5.3 Payment Method Gaps

Credit card limitations necessitate support for alternative payment methods (digital wallets, bank transfers, M-pesa).

5.4 Mobile-First Requirements

Emerging markets often exhibit mobile-first usage patterns, yet many systems remain desktop-optimised.

5.5 Regulatory and Compliance Gaps

Local laws on data, tax, and consumer protection require tailored compliance strategies.

6. Synthesis and Research Implications

The literature reveals three key themes:

1. Architecture matters: Marketplace aggregators outperform fragmented direct connects.
2. Framework choice: Java Spring provides scalability, integration, and flexibility.
3. Local adaptation: Addressing content, language, payment, mobile, and regulatory gaps is critical for adoption.

7. Conclusion

Flight booking systems have evolved from legacy GDS platforms to modern marketplace aggregators. Java Spring frameworks provide a robust foundation for next-generation systems. However, persistent local adoption gaps highlight the need for context-aware design. Balancing global scalability with local adaptability remains the central challenge and opportunity for developers of future flight booking platforms.

CHAPTER THREE — MATERIALS AND METHODS 3.1 Proposed Work

The project adopts a structured development methodology involving requirement analysis, architectural design, implementation, and testing. Modular development principles ensure scalability, maintainability, and future extensibility.

3.2 Requirement Analysis Software Requirements

- Java Spring Boot framework
- MySQL database system
- REST API tools
- Integrated development environment

Hardware Requirements

- Standard development workstation
- Network connectivity
- Server environment for deployment testing

3.3 System Design

The Flyte system employs a client–server architecture comprising flight management, booking services, and administrative modules. Database design follows relational modeling principles to ensure consistency and integrity. Functional workflows support reservation processing, authentication, and transaction management.

3.4 Testing Procedures

System validation includes module testing, integration testing, and simulated user scenarios. Performance evaluation measures system responsiveness, reliability, and usability, enabling iterative refinement.

REFERENCES

(To be completed following institutional citation guidelines.)

APPENDICES

- System diagrams
- Database schemas
- Interface screenshots
- Test case